

IMPLEMENTACIÓN DE UNA PLATAFORMA DIGITAL PARA LA GESTIÓN EFICIENTE DE CITAS MEDICAS.

Integrantes:

Juan Diego Cano Jaimes

Xiomara Peña Tapias

Objetivo General

- Diseñar un sistema digital de agendamiento para optimizar la asignación de citas y reducir los tiempos de espera de los usuarios.

Objetivos Específicos

- Identificar las fallas actuales en el proceso de reserva de turnos mediante encuestas o entrevistas a los usuarios.
- Desarrollar una plataforma web o móvil que permita a los clientes programar, cancelar o reprogramar sus citas de forma autónoma.

Informe de Proyecto: Sistema de Gestión Médica UTS

Se presenta la documentación técnica del Sistema de Gestión Médica UTS, diseñado bajo el patrón de arquitectura Modelo-Vista-Controlador (MVC) utilizando Java Desktop (Swing) y SQLite como motor de base de datos relacional

Arquitectura del Sistema

El sistema implementa el patrón arquitectónico MVC, lo que garantiza una separación clara de responsabilidades, facilitando el mantenimiento y la escalabilidad del software:

- **Modelo (Model):** Compuesto por las clases de acceso a datos (CitaDAO, DoctorDAO, FacturaDAO, PacienteDAO) y las entidades de negocio (Persona). Administran la lógica de datos y las transacciones con la base de datos.
- **Vista (View):** Representado por las interfaces gráficas desarrolladas en Java Swing (LoginView, MainMenuView, PacienteCRUDView,

DoctorCRUDView, CitaCRUDView, FacturaCRUDView. No contienen lógica de negocio ni de persistencia.

- **Controlador (Controller):** Clases intermediarias (LoginController, MainMenuController, PacienteController, DoctorController, CitaController, FacturaController) que capturan las acciones del usuario en las vistas, manipulan los DAO correspondientes y actualizan las interfaces gráficas

Modelo de la Base de Datos

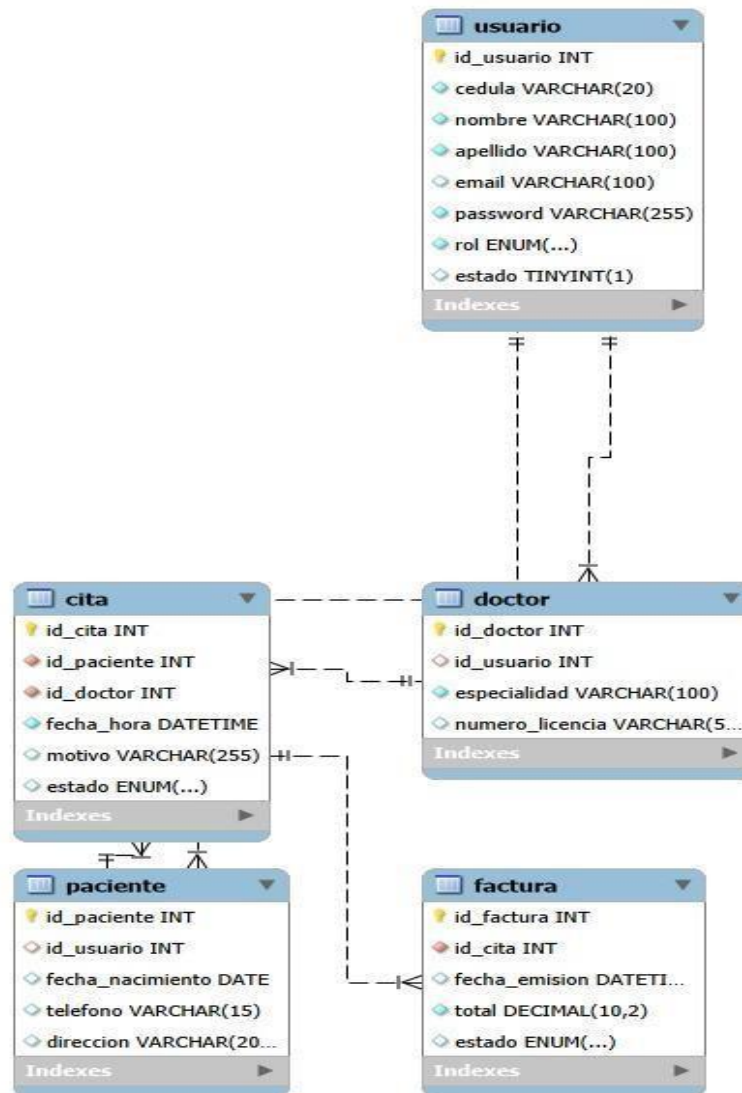
La persistencia se realiza mediante una base de datos relacional en SQLite denominada bd_personas.db. El diseño cuenta con una estrategia de herencia física (Table-per-Type) donde la información común se centraliza en una tabla raíz

Diagrama de Entidad-Relación

A nivel conceptual y físico, la estructura se compone de las siguientes entidades interconectadas:

- **usuario:** Tabla maestra que unifica las credenciales de acceso y datos básicos (cédula, nombre, apellido, email, password, rol y estado).
- **paciente / doctor:** Tablas que extienden a la tabla usuario mediante una relación de uno a uno (1:1), utilizando id_usuario como llave foránea con borrado en cascada (ON DELETE CASCADE).
- **cita:** Entidad intermedia que gestiona el agendamiento médico, vinculando de forma multivalorada (1:N) a un paciente y a un doctor.
- **factura:** Entidad dependiente que registra el cobro financiero asociado de manera exclusiva (1:1 o 1:N) a una cita médica previamente programada.

Estructura DDL (Data Definition Lagunaje)



```

CREATE TABLE IF NOT EXISTS usuario (
    id_usuario INTEGER PRIMARY KEY AUTOINCREMENT,
    cedula TEXT UNIQUE NOT NULL,
    nombre TEXT NOT NULL,
    apellido TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    rol TEXT NOT NULL, -- 'Administrador', 'Empleado' (antes Secretaría), 'Doctor', 'Paciente'
    estado INTEGER DEFAULT 1
);
    
```

```

-- 2. Tabla: Pacientes (Relacionada uno a uno con Usuario)
CREATE TABLE IF NOT EXISTS paciente (
    id_paciente INTEGER PRIMARY KEY AUTOINCREMENT,
    id_usuario INTEGER NOT NULL,
    fecha_nacimiento TEXT NOT NULL,
    telefono TEXT,
    direccion TEXT,
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON DELETE CASCADE
);

-- 4. Tabla: Citas Médicas (Une a un Paciente con un Doctor)
CREATE TABLE IF NOT EXISTS cita (
    id_cita INTEGER PRIMARY KEY AUTOINCREMENT,
    id_paciente INTEGER NOT NULL,
    id_doctor INTEGER NOT NULL,
    fecha_hora TEXT NOT NULL, -- Formato: AAAA-MM-DD HH:MM
    motivo TEXT,
    estado TEXT DEFAULT 'Pendiente', -- 'Pendiente', 'Completada', 'Cancelada'
    FOREIGN KEY (id_paciente) REFERENCES paciente(id_paciente) ON DELETE CASCADE,
    FOREIGN KEY (id_doctor) REFERENCES doctor(id_doctor) ON DELETE CASCADE
);
-- 5. Tabla: Factura (Registra el cobro asociado a una Cita)
CREATE TABLE IF NOT EXISTS factura (
    id_factura INTEGER PRIMARY KEY AUTOINCREMENT,
    id_cita INTEGER NOT NULL,
    fecha_emision TEXT NOT NULL,
    total REAL NOT NULL,
    estado TEXT DEFAULT 'Pendiente', -- 'Pagada', 'Pendiente'
    FOREIGN KEY (id_cita) REFERENCES cita(id_cita) ON DELETE CASCADE
);

```

Explicación del Código Fuente

Conexión (Conexion.java)

Aplica el patrón Singleton para administrar una única instancia de conexión hacia el archivo local de SQLite a través del driver org.sqlite.JDBC, optimizando la apertura y cierre de hilos hacia la base de datos.

Componente de Autenticación (LoginController.java)

Valida las credenciales ingresadas (email y password) mediante un PreparedStatement parametrizado para mitigar ataques de inyección SQL. Tras el éxito de la consulta, recupera el atributo rol para instanciar el menú principal (MainMenuView) con los permisos específicos requeridos.

Módulo de Pacientes y Médicos (PacienteController / DoctorController)

Ambos controladores implementan la interfaz ActionListener para gestionar operaciones CRUD. Debido a la arquitectura de la base de datos, el flujo de inserción maneja transacciones explícitas (setAutoCommit(false):

1. Se registra primero la entidad general en la tabla usuario.
2. Se recupera la llave primaria generada mediante Statement.RETURN_GENERATED_KEYS.
3. Se insertan los atributos especializados en las tablas paciente o doctor utilizando el ID recuperado.
4. Si alguna de las operaciones falla, se ejecuta un rollback(); en caso de éxito, se aplica el commit() para garantizar la atomicidad (ACID).

Asignación de Citas (CitaController.java)

Encargado de la lógica de negocio para la reserva de turnos. Destaca el método cargarDesplegables(), el cual pobla los componentes JComboBox de la interfaz gráfica extrayendo información combinada desde los DAOs de soporte.

Posteriormente, traduce el índice seleccionado en la UI con los identificadores reales (id_paciente e id_doctor) almacenados en colecciones internas del tipo List<Integer>.

Facturación (FacturaController.java)

Automatiza el proceso de cobro enlazando directamente el identificador único de la cita médica (id_cita) con un valor numérico de precisión flotante (double total), permitiendo el rastreo del estado contable ('Pagada' o 'Pendiente').

Inicialización del Entorno

El sistema cuenta con un script de inserción inicial (Seed) para permitir el acceso directo en fases de prueba y despliegue técnico:

sql

```
INSERT OR IGNORE INTO usuario (cedula, nombre, apellido, email, password, rol, estado)
```

```
VALUES ('123456', 'Admin', 'UTS', 'admin@uts.edu.co', 'admin123', 'Administrador', 1);
```

Usa el código con precaución.

Para poner en marcha la aplicación, se debe compilar y ejecutar la clase principal Main.java, la cual inicializa de forma inmediata la interfaz de autenticación del usuario (LoginView).

Conclusiones

La automatización del agendamiento disminuye los errores en la asignación de turnos y mejora la organización interna, ofreciendo una opción digital para aumentar la satisfacción del cliente al permitirle reservar su cita en cualquier momento, sin depender de llamadas telefónicas.